

This exam consists of 17 questions.

Points are indicated for each question and broken down to each part.

Question 1: Defining Software Engineering (4 points)

Define the term *Software Engineering* as discussed in the lecture. State whether software engineering is considered a *true engineering discipline*. Provide one argument supporting this view and one argument against it.

Marking:

Source: Lo1_intro_and_crisis, Slides 10-15, 22-26, 29.

- **1 point** for a correct definition of Software Engineering (e.g., "A discipline within Computer Science; a set of methodologies, techniques, and tools to build high-quality software in a cost-effective way," or "the application of a systematic, disciplined, quantifiable approach...").
- **1 point** for stating the general conclusion from the lecture (e.g., "Not really an engineering discipline – but has a potential to become one!"). Enough to state that it is no engineering discipline.
- **1 points** for a supporting argument (e.g., It applies scientific knowledge like algorithms and data structures; it uses a systematic approach with tools and processes; it aims to solve practical problems).
- **1 points** for a contradicting argument (e.g., It lacks a codified body of fundamental theory; much knowledge is tacit and relies on 'virtuoso' problem solving; it is rapidly evolving, which makes codification difficult; it is a label for current practices rather than a science-based discipline).
- note that the question could be misunderstood: "one argument supporting this view" could refer to being an engineering discipline or to not being one. In any case, if the answer refers to differences like cost of change, these are not valid pro/con arguments and will not receive marks.

Question 2: The Software Crisis (6 points)

List and briefly describe two primary reasons that contributed to the *Software Crisis* and briefly describe two symptoms identified in the lecture. State and justify whether software lifecycle models address the reasons or the symptoms.

Marking:

Source: Lo1_intro_and_crisis, Slides 35-39, 49-52.

- **2 points (1 per reason)** for listing and briefly describing two primary reasons (or others from the lecture), e.g.
 - **Rising demand for software:** As hardware became cheaper and more powerful, demand for complex software to run on it grew rapidly.
 - **Increase in product complexity:** Systems became larger and more intricate, increasing the development effort required.
 - **Slow increase in development productivity:** The productivity of developers did not keep pace with the explosion in demand and complexity.
- **2 points (1 per symptom)** for listing and briefly describing the symptoms from the lecture, e.g., dollar/time overrun, system never used as is. Also allow bugs as one symptom (this would mean that re-work is needed).
- **2 points** for providing a statement and justification (likely they address the symptoms, e.g., by requirements engineering, but also reasons, e.g., better organization)

Question 3: Waterfall vs. Spiral Model (4 points)

Describe the core philosophies of the Waterfall model and the Spiral model. For each lifecycle model, describe and justify the type of software project for which it is most suitable.

Marking:

Source: LO2_lifecycle_models, Slides 23-25, 30-36.

- **1 points** for describing the Waterfall model's philosophy (e.g., A linear, sequential, document-driven process where each phase must be fully completed before the next begins. It is plan-driven with very limited iteration).
- **1 points** for describing the Spiral model's philosophy (e.g., An iterative, risk-driven process that combines elements of prototyping and the Waterfall model. Development proceeds in several rounds or spirals, with each round focusing on risk analysis and mitigation).
- **1 points** for identifying the suitable project type for the Waterfall model and justifying it (e.g., Suitable for projects with stable, well-understood requirements and technologies, where the domain is well-known. The linear structure works well when there is little uncertainty).
- **1 points** for identifying the suitable project type for the Spiral model and justifying it (e.g., Suitable for large, complex, and high-risk projects where requirements are unclear or expected to change. The risk-driven approach allows for early identification and mitigation of major uncertainties).

Question 4: Prototyping Models (6 points)

Compare *Evolutionary Prototyping* and *Throwaway Prototyping* (one commonality, one difference in the approaches). For each individual approach, state one differentiating advantage and one differentiating disadvantage.

Marking:

Source: LO2_lifecycle_models, Slides 39-43.

- **2 point** for clearly distinguishing between the two models by providing a commonality (1 mark, e.g., both create prototypes) and one difference (1 mark, e.g., prototype is kept vs. thrown away)
- **2 points** for Evolutionary Prototyping:
 - **Advantage (1 point):** Immediate user feedback; low risk of developing the wrong system.
 - **Disadvantage (1 point):** Difficult to plan (time/cost); can degrade into an unstructured code-and-fix approach.
- **2 points** for Throwaway Prototyping:
 - **Advantage (1 point):** Can be done quickly; no risk of carrying technical debt from the prototype into the final system.
 - **Disadvantage (1 point):** User confusion (mistaking the prototype for the final system); cost and time of building a prototype that will be discarded.

Question 5: Importance of Requirements Engineering (3 points)

Explain the relation between *Software Quality* and *fitness for purpose*. Explain how this makes Requirements Engineering a critical activity for the success of a software development project.

Marking:

Source: LO3_requirements_engineering, Slides 4, 5, 8.

- **1 point** for defining "Software Quality" as "fitness for purpose" (e.g., Quality is not an intrinsic property of the software itself but is a function of how well the software serves its intended purpose in its context).
- **1 points** for explaining why RE is critical (e.g., "If we don't understand the requirements, we're going to build the wrong system." It is the process of identifying and fixing errors as early as possible, which is crucial because the cost of correcting errors grows significantly in later stages).
- **1 points** for explaining how RE addresses this definition (e.g., RE is the primary activity for discovering and defining the software's "purpose." By establishing the services the customer expects and the constraints, RE ensures that the development effort is aligned with what will be considered a "fit" or high-quality solution by the stakeholders).

Question 6: Stakeholders and Conflicting Goals (6 points)

For a new mobile banking application, identify two distinct *internal stakeholders* and two distinct *external stakeholders*. For each of the four stakeholders, state one relevant concern they might have. Finally, provide one example of a potential *conflicting goal* between any two of the stakeholders you identified.

Marking:

Source: LO3_requirements_engineering, Slides 11, 12.

- **2 point** (0.5 points per stakeholder/role, 0.5 points per concern matching the role) for identifying two distinct internal stakeholders with a relevant concern each (e.g., **Marketing Team:** wants quick feature releases to attract new users. **Legal Department:** wants to ensure all features comply with strict financial regulations, which may slow down development).
- **2 point** (0.5 points per stakeholder/role, 0.5 points per concern matching the role) for identifying two distinct external stakeholders with a relevant concern each (e.g., **Customers/Users:** want a simple, fast, and easy-to-use interface. **Government Regulators:** want robust security measures and detailed audit trails).
- **2 points** for a clear and plausible example of a conflicting goal. The example must be specific. (e.g., "The **Marketing Team's** goal of a simplified, one-click fund transfer feature conflicts with the **Legal Department's** goal of implementing a mandatory two-factor authentication process for all transactions to meet security regulations. The former prioritizes user convenience, while the latter prioritizes security and compliance.").
- **Note:** If 'internal' or 'external' is not specified, consider either 'internal' or 'external' (only one) based on intuition. If the bank is developing the software internally, then the bank manager and bank staffs can be considered internal stakeholders.

Question 7: Functional vs. Non-Functional Requirements (6 points)

Clearly distinguish between *Functional* and *Non-Functional* requirements. Provide two examples of verifiable non-functional requirements for an online video streaming service. For each example, explain why it is non-functional and what makes it verifiable.

Marking:

Source: LO3_requirements_engineering, Slides 30-34.

- **2 points** for a clear distinction (e.g., Functional requirements define *what* the system does—its functions, features, and computations. Non-functional requirements define the *qualities* of the system—how well it performs its functions, such as its performance, reliability, or usability).
- **2 points** for the first verifiable non-functional requirement and explanation.
 - **1 point** for example (0.5 for non-functional and 0.5 for verifiable).
 - **Example:** "The system shall start video playback within 3 seconds of the user pressing the play button for 95% of all requests on a broadband connection."
 - **1 point** for explanation (0.5 for explaining non-functional aspects and 0.5 for explaining why/how verifiable aspects work).
 - **Explanation:** This is non-functional because it describes a quality (performance), not a function. It is verifiable because the 3-second threshold and 95% success rate can be objectively measured during testing.
- **2 points** for a second, distinct verifiable non-functional requirement and explanation.
 - **1 point** for example (0.5 for non-functional and 0.5 for verifiable).
 - **Example:** "The system must be available 99.9% of the time during peak hours (6 PM - 11 PM local time)."
 - **1 point** for explanation (0.5 for explaining non-functional aspects and 0.5 for explaining why/how verifiable aspects work).
 - **Explanation:** This is non-functional because it describes a quality (reliability). It is verifiable through system monitoring and uptime logs.

Question 8: Ambiguity in Natural Language Requirements (6 points)

Define *ambiguity* in the context of software requirements. Provide a specific example of an ambiguous requirement for an online forum system. Finally, explain a method from the lecture and how it may mitigate ambiguity in your example requirement.

Marking:

Source: LO4_requirements_representations, Slides 6, 10-12, 18.

- **2 points** for a clear definition of ambiguity (e.g., "The quality of being open to more than one interpretation." The explanation could note that this can cause readers and writers to have different, subconscious interpretations of the same requirement).
 - **-1 point** if not related to software requirements in the text
 - **-1 point** if confused with incompleteness or being imprecise (these are fundamentally different concepts!)
- **1 point requirement, 1 point why ambiguous** for a specific, plausible example of an ambiguous requirement (e.g., "The system shall remove old posts." This is ambiguous because 'old' is not defined. Does it mean posts older than a month? A year? Posts with no recent activity?).
- **1 point for method, 1 point how it mitigates here** for explaining one mitigation method, e.g.,
 - **Rephrasing/Splitting:** The requirement could be rewritten for clarity, such as "The system shall archive posts that have had no new comments for more than 180 days."
 - **Using Controlled Natural Language (CNL):** This involves using a restricted subset of natural language with a defined grammar and vocabulary to reduce ambiguity. For example, using a sentence pattern like "WHEN [condition], THE SYSTEM SHALL [action]" forces precision.
 - **writing user-stories**

Question 9: Use Case Diagram Relationships (4 points)

Explain the meaning and purpose of the <<include>> and <<extend>> relationships in UML Use Case Diagrams. For each relationship, provide a clear and distinct example use (excerpt of a Use Case Diagram) in the context of a food delivery application.

Marking:

Source: LO4_requirements_representations, Slides 45, 46.

- **1 point** for explaining <<include>> (e.g., It specifies that the behavior of one use case (the included one) is a mandatory part of another (the base use case). The base use case is incomplete without the included one).
- **1 point** for a clear example of <<include>> (e.g., The "Place Food Order" use case must *always* involve payment, so it would <<include>> a "Process Payment" use case).
- **1 point** for explaining <<extend>> (e.g., It specifies that one use case (the extending one) provides optional functionality that may be inserted into another (the base use case) only under certain conditions, defined at an extension point).
- **1 point** for a clear example of <<extend>> (e.g., The "Place Food Order" use case could be *optionally* extended by an "Apply Discount Code" use case, which only runs if the user decides to enter a code).

Question 10: Model Criteria (6 points)

Define what a *model* is in the context of software engineering based on the three core criteria a representation must satisfy to be considered a model, as discussed in the lecture. Illustrate each criterion on a single software model.

Marking:

Source: LO5_software_models, Slides 14, 15, 20.

- **3 point** for defining a model based on: Mapping, Reduction (Abstraction), and Pragmatic.
 - 1 instead of 3 points if criteria are only listed but not explained
- **3 points (1 per criterion)** for briefly illustrating each aspect on a single, common software model (remove 1 mark if the model is not a software model, e.g., a map as a model of a city). 0 points if the example is not a model:
 - **Mapping Criterion:** The model is based on an original object or phenomenon (a referent).
 - **Reduction Criterion (Abstraction):** The model only reflects a relevant selection of the original's properties; some information is intentionally omitted.
 - **Pragmatic Criterion:** The model must be usable in place of the original for some specific purpose; it must be useful.

Question 11: Elements of a Modeling Language (6 points)

List and briefly describe the three fundamental elements that define a modeling language. Illustrate these three elements using the example of UML Class Diagrams.

Marking:

Source: LO5_software_models, Slides 51-53, 61-62.

- **3 points (1 per element)** for listing and describing the three elements (only 1 point in total if all 3 elements are listed but not described):
 - **Syntax:** Defines the set of allowed expressions and their structure. It can be visual (e.g., boxes, lines) or textual, with rules for composition.
 - **Semantic Domain:** Provides the meaning for expressions by specifying the concepts that exist in the universe of discourse (e.g., the "things" the language can talk about).
 - **Semantic Mapping:** Maps each syntactically correct expression to its meaning within the semantic domain.
- **3 points (1 per element)** for illustrating with UML Class Diagrams:
 - **Syntax:** The visual syntax consists of boxes for classes, lines for associations, and text for names and attributes, with rules about how they can be connected.
 - **Semantic Domain:** The semantic domain consists of sets of objects, their attribute values, and the links between them (often represented by object diagrams).
 - **Semantic Mapping:** The mapping specifies that a class diagram is valid for a set of objects (an object diagram) if those objects are valid instances of the classes and their links respect the associations defined in the class diagram.

Question 12: Language Selection Criteria (4 points)

List two distinct criteria for selecting a programming language for a software project as discussed in the lecture. For each criterion, provide a brief explanation of its importance.

Marking:

Source: LO6_Implementation, Slide 19.

- **1 point** per criterion
- **1 point** for its importance

Question 13: Programming Style and Readability (4 points)

A program is read much more often than it is written. Provide two specific examples of naming conventions discussed in the lecture (e.g., for classes, methods, or constants) and explain how each one improves code readability.

Marking:

Source: Lo6_Implementation, Slides 24, 35-37.

- **2 points (1 per example)** for two specific naming conventions:
 - **Classes:** Noun, first letter uppercase (e.g., 'Account').
 - **Methods:** Verb, first letter lowercase (e.g., 'checkAvailability()').
 - **Constants:** All capital letters, words separated by underscores (e.g., MAX_WIDTH).
- **2 points (1 per rationale)** for explaining how each convention improves readability:
 - **Classes:** Capitalization makes it instantly clear that an identifier refers to a type (a blueprint for objects), distinguishing it from variables or methods.
 - **Methods:** Using a verb clearly indicates that the identifier represents an action or operation, making the code read more like a set of instructions.
 - **Constants:** All-caps notation signals that the value is fixed and should not be changed, preventing confusion and accidental modification.

Question 14: Fundamental Testing Concepts (6 points)

Briefly define the terms *Error*, *Failure*, and *Fault* according to the IEEE terminology discussed in the lecture. Provide a single, clear example from a web application (e.g., an e-commerce site) that illustrates the relationship between all three concepts.

Marking:

Source: Lo7_Testing, Slides 5-10.

- **3 points (1 per term)** for correctly defining each term:
 - **Failure:** Observable incorrect behavior of the system from the user's perspective.
 - **Fault (or Bug/Defect):** An incorrect piece of code or system state that can lead to a failure.
 - **Error:** The human action or misconception that resulted in a fault (e.g., a typo, a logical mistake).
- **3 points** for a clear, single example that correctly links all three concepts. The example must be distinct from the one in the slides. (e.g., "On an e-commerce site, a developer makes an **Error** by using '¿' instead of '¿=' when checking for available stock. This creates a **Fault** in the inventory checking code. A user trying to buy the last available item (stock=1) sees an 'out of stock' message; this incorrect behavior is the **Failure**.").

Question 15: Practical White-Box Testing and Mutation Testing

Consider the following Java code snippet:

```
1 public int calculate(int a, int b) {  
2     int r = 0;  
3     if (a > 5 && b == 0) {  
4         r = a;  
5     } else {  
6         r = a + b;  
7     }  
8     if (r < 0) {  
9         r = 0;  
10    }  
11    return r;  
12 }
```

(a) (3 points) Provide a minimal set of test cases to achieve maximal branch coverage.

Marking:

Source: LO7_Testing, Slides 85-96, 128-132.

Test cases should usually have input and expected output (we will also accept if only the input is given).

- A minimal set of test cases for 100% branch coverage requires covering the true/false paths of both 'if' statements. (2 points for a correct, minimal set). A minimal set requires 2 test cases. For example: (-6, 0, 0) and (6, 0, 6)
- **-1 if not minimal, -1 for each missing branch (4 branches in total)**

- (b) (3 points) Briefly explain what a *live mutant* is in Mutation Testing. Propose one simple mutation to the original code snippet that your test suite from part (a) does **not** kill, and explain why.

Marking:

- **1 point** for defining a "live mutant" (e.g., A mutant that is not killed by any test case in the test suite, meaning its behavior is identical to the original program for all test inputs, either because it's equivalent or the test suite is inadequate). 0 points if live is described as *not killed* but neither are explained.
- **1 point** for proposing a plausible mutation and **1 point** for explaining why it would survive.
Example:
 - **Mutation:** Change 'result = a + b;' to 'result = b + a;'
 - **Explanation:** This mutant is functionally equivalent to the original code because addition is commutative. No test case can ever distinguish its output from the original program, so it will always remain a "live mutant."

Question 16: Extreme Programming (XP) Practices (6 points)

Briefly explain three distinct practices of Extreme Programming (XP) from the lecture. For each chosen practice, describe its main goal.

Marking:

Source: Lo8_AgilePrinciples, Slides 18, 27-31, 35-38.

- **2 points** for the first chosen practice:
 - **1 point** for explaining the practice (e.g., **Pair Programming:** All code is written by two programmers sharing one machine, with one "driving" (typing) and the other "navigating" (observing/reviewing)).
 - **1 point** for its main goal (e.g., To improve code quality and design through continuous review and collaboration).
- **2 points** for a second, distinct chosen practice:
 - **1 point** for explaining the practice (e.g., **Refactoring:** Modifying the internal structure of existing code to improve its design without changing its external functionality).
 - **1 point** for its main goal (e.g., To improve software maintainability, extensibility, and to remove "code smells").
- **2 points** for the third, distinct chosen practice:
 - **1 point** for explaining the practice (e.g., **on-site customer:** (or a proxy who truly understands business needs) is physically present with the development team during the project).
 - **1 point** for its main goal (e.g., To ensure continuous, immediate feedback and close collaboration between developers and customers — minimizing misunderstanding and rapid development).
- **Note: Only 1 point instead of 3** if three practices are listed but not described.

Question 17: Anti-Patterns vs. Code Smells (5 points)

Briefly explain what an "Anti-Pattern" is and how it differs from a "Code Smell." Provide one example of an anti-pattern from the lecture and explain the common problem it tries to solve and the negative consequences it introduces.

Marking:

Source: Lo9_CodeQuality, Slides 20, 29-37.

- **2 points** for the explanation and difference (e.g., An Anti-Pattern is a commonly used but flawed solution to a recurring problem. The key difference is that an anti-pattern is a bad *solution* or *process*, whereas a code smell is a problematic *structure* within the code itself).
- **1 point** for one example of an anti-pattern (e.g., "FloatingPointCurrency," "ParsingHTML-WithRegex," "ZeroMeansNull").
- **2 points** for explaining the problem and consequences of the chosen example:
 - **Example: FloatingPointCurrency**
 - **Problem it solves:** It seems like a simple way to store an amount of money (e.g., 1.23 for €1.23).
 - **Negative Consequences:** Standard floating-point types cannot accurately represent all decimal fractions, leading to accumulating rounding errors over time, which is unacceptable for financial calculations.